

StarfishOS: Revisiting Single System Image on CXL with State-Partitioned Microkernel

Submission #906

Abstract

Single System Image (SSI) offers an attractive OS abstraction that enables single-machine applications to transparently execute on multiple machines and utilize pod-wide resources. The emergence of Compute Express Link (CXL) shifts cross-machine synchronization from message passing to coherent shared memory, potentially removing SSI’s long-standing performance barrier. However, placing state in slow CXL memory degrades performance and makes the system vulnerable to partial failures, since inconsistent CXL-resident state may block or crash the whole system.

We observe that microkernels’ small kernel-state footprint and capability-based state management make OS state explicit enough to partition and minimize what resides in CXL. We therefore propose the *state-partitioned microkernel* and realize it in StarfishOS, the first OS-level SSI to use CXL shared memory while preserving both efficiency and OS-level partial-failure resilience through minimal, recoverable CXL state. Evaluation shows that applications can transparently scale to the full pod on StarfishOS with up to ~~4×~~ and ~~2×~~ speedup compared to their distributed and CXL-optimized versions. Moreover, StarfishOS utilizes all stranded CPU resources by running applications across machines.

1 Introduction

Compute Express Link (CXL) [3] connects rack-scale machines into a *CXL pod* [63], where all machines access a shared memory pool through load/store [4]. This shared-memory primitive replaces inter-machine messaging with load/store access, enabling much cheaper cross-machine synchronization. Prior work has already demonstrated the potential of this model. For example, CXL-SHM [69] builds a MapReduce program from scratch to exchange shuffle data through CXL shared memory to reduce execution time. Tigon [35], on the other hand, reorganizes database data structures to be CXL-aware, avoiding two-phase commit [30] and improving OLTP throughput by 18.5×

Despite their performance gains, a broader question arises: to benefit from CXL, is it necessary to re-design and re-implement for every application, especially those with large and hard-to-modify codebases?

To answer this question, we revisit OS-level single system image (SSI) [24, 27, 57, 62] as a way to enable existing single-machine shared-memory applications to transparently benefit from CXL. SSI exposes a CXL pod as one logical machine with a single address space, PID namespace, file namespace, and device namespace. Under SSI, the OS transparently schedules application threads pod-wide and

routes all cross-machine operations through CXL load/store, without any application redesign.

Building SSI on CXL, however, is non-trivial. At one extreme, the system can keep all OS and application state in local DRAM and communicate across machines via message passing, as traditional SSI systems do [24, 27, 62]. However, this design fails to exploit CXL’s shared memory semantics and remains inefficient due to message-passing [22, 41, 47, 58]. At the other extreme, the system can place all OS and application state on CXL, so the pod behaves like a shared-memory machine on which existing OSes and applications run unmodified. While simple and fully exploiting CXL’s shared memory semantics, this design faces two obstacles:

(1) *Efficiency*: CXL memory is slower than local DRAM: 2–5× access latency and 20%–80% bandwidth of local DRAM [54, 59, 69]. Placing all kernel and application state in CXL causes unacceptable overhead for kernel and applications (up to 3.4× slowdown on our platform, as shown in Figure 13).

(2) *Partial failure*: in CXL pod, host machines and CXL memory modules have separate power supplies and can fail independently [69, 73]. A host crash may leave CXL-resident locks unreleased or leave data structures in inconsistent states; surviving machines that observe them may block or crash, turning a single-host failure into a pod-wide outage.

Key insight. Both obstacles stem from the same cause: placing too much OS state on CXL. *State partitioning* resolves both simultaneously, i.e., keeping only the minimal coordination state in CXL, and leaving everything else in private DRAM for efficiency. Because this shared state is small, it can be engineered for partial-failure resilience, so a host crash need not block or crash the surviving machines.

Realizing this in practice requires a *clear state-management abstraction*: the OS must explicitly identify each state object, its type, and its dependency edges. Monolithic kernels generally lack such an explicit model—their state is spread across tightly coupled kernel subsystems with implicit references—so fine-grained partitioning and placement become impractical [34, 56]. Microkernels, in contrast, are suited for state partitioning. First, their kernel-space state is small, enumerable, and individually manageable [38, 68]. Second, microkernels’ capability-based state management tracks each OS object as a first-class entity with explicit type and capabilities for access control, enabling per-object placement decisions [38, 68].

Contributions. We present StarfishOS, the first OS-level SSI for CXL pods with a *state-partitioned microkernel*. Each layer of the OS stack, including kernel-space modules (IPC,

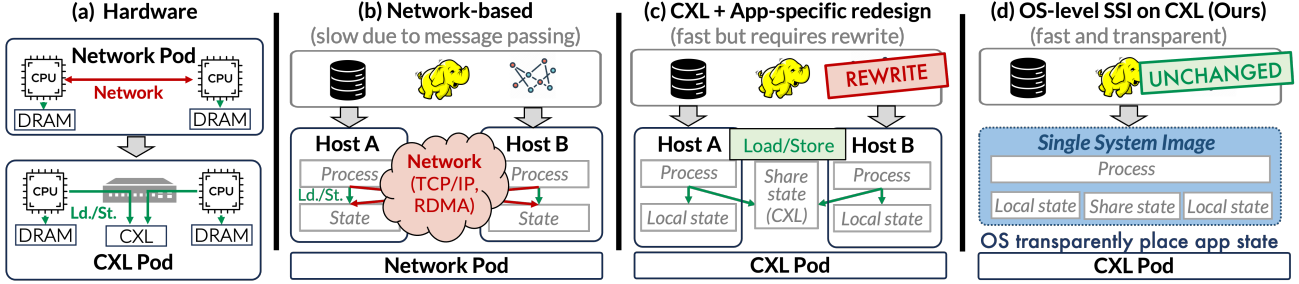


Figure 1: Compare network-based and app-specific redesign approaches with our OS-level SSI for CXL Pod.

scheduling, memory management), system services (filesystems, process managers, etc.), and applications, faces a distinct state-sharing challenge; StarfishOS addresses each with a tailored partitioning and OS-level resilience strategy.

For efficiency, StarfishOS applies the same partitioning rule across the OS stack: only state needed for cross-machine coordination resides in CXL, while locality-sensitive state remains in private DRAM. In the kernel, this means sharing only the data structures that implement inter-machine functionality, such as IPC queues, scheduling queues, and the CXL allocator. In system services, StarfishOS designs collaborative system service instances to keep only global coordination metadata in CXL and leave service state inside per-machine service instances. For applications, StarfishOS uses OS-visible semantics to partition application state into objects and place each object in either CXL or private DRAM. StarfishOS also introduces a page-fault-driven mechanism that migrates heap pages to CXL only on demand. This makes CXL usage small, and aligned with actual state sharing rather than with putting everything on CXL.

For handling partial failures, StarfishOS targets OS resilience rather than transparent application recovery. A machine crash should therefore resemble the loss of processes on a single machine, without blocking or corrupting the rest of the pod. To achieve this, StarfishOS makes its CXL-resident kernel coordination state recoverable with lock-free, detectably recoverable data structures [16, 29], keeps most service state outside CXL so failed service instances can be re-created via micro-reboot, and leaves application-level recovery to the applications’ own mechanisms. As a result, surviving machines can continue using both the OS and the shared CXL state after other machines fail.

Evaluation. We evaluate StarfishOS’s two main benefits for existing single-machine applications. First, StarfishOS transparently scales unmodified shared-memory applications to the full pod: applications such as Phoenix [49], DBx1000 [5], and GeminiGraph [72] scale to 8 machines with 2.0–6.0× speedup and zero code changes. For Phoenix and GeminiGraph, StarfishOS outperforms their distributed versions by 4.2× and 2.0×, respectively. For DBx1000, StarfishOS achieves 4× speedup compared to Tigon [35]. Second, StarfishOS enables applications to utilize stranded CPU resources across machines. In traditional OSes, a multithreaded application

must be bin-packed onto one machine, so fragmented CPUs across machines cannot be used. In contrast, StarfishOS lets applications use all idle CPUs across machines.

In summary, this paper makes the following contributions:

- We revisit SSI in the CXL era and present the first OS-level SSI that uses CXL’s shared-memory primitive for cross-machine synchronization.
- We propose the state-partitioned microkernel to achieve efficient SSI with OS-level partial-failure resilience.
- We support automatic scaling of existing single-machine applications across multiple machines and improve pod-wide resource utilization.

2 Revisiting Single System Image with CXL

CXL yields application performance gains. Compute Express Link (CXL) [3] lets multiple hosts access shared memory with load/store instructions, much as a single machine accesses local DRAM. Such a setup is called a CXL pod [63] and is shown in Figure 1.

Recent work shows that using this shared memory (Figure 1 (a)) instead of message passing (Figure 1 (b)) can substantially improve application performance. For database system, Tigon [35] keeps cross-host active items in CXL and synchronizes concurrent accesses with load/store/atomics on shared memory. This design avoids the costly message exchanges of two-phase commit (2PC) [30] for multi-partition transactions, which can take 300μs–2ms per operation, and achieves up to 18.5× higher TPC-C [12] throughput than message-passing-based designs. CXL-SHM [69] reports similarly large benefits for MapReduce-style workloads by placing shuffle data in shared CXL memory. This design allows workers on different machines to access shuffle data directly with load/store, avoiding message passing and greatly reducing the execution time of WordCount and KMeans.

Current gains require application-specific redesign.

So far, the largest gains have come from *application-specific redesign*. For example, Tigon [35] introduces over 5,000 LoC of changes to an 18,000-LoC codebase to restructure database data structures, migrates items across DRAM/CXL. Despite this effort, Tigon remains specialized to locking-based concurrency control, including 2PL [30] and next-key locking [30]. CXL-SHM [69] exhibits a similar pattern: although

it reports comparable gains, it rewrites MapReduce from scratch to accommodate its pass-by-reference design.

Our choice: OS-level SSI. We therefore ask whether applications can gain CXL’s benefits *without per-application redesign*, and whether a single approach can apply to a *broad class* of applications. Since decades of experience show that single-machine, multithreaded applications already exploit shared-memory semantics effectively, we explore a different path: an *OS-level single system image (SSI)* that makes a CXL pod appear as a large shared-memory machine. SSI makes a CXL pod appear as a large shared-memory machine, and brings two direct advantages:

(1) *Auto-scale existing programs to transparently benefit from CXL*: legacy single-machine applications, whether single- or multi-process, can be auto-scaled to run on any CPU in the CXL pod with zero code changes. As shown in Figure 1 (d), the OS transparently schedules a process’s threads across the pod and places its state on DRAM/CXL memory on demand, so cross-node communication transparently becomes load/store rather than message passing.

(2) *Better resource utilization and failure handling*: with a single view of pod-wide resources provided by SSI, applications can transparently use remote devices such as SSDs/NICs. Prior work reports that pooling devices across 8 hosts can increase device utilization by 46–81% [71]. Applications can also seamlessly fall back to remote devices when local ones fail, rather than waiting for repair [70, 71].

SSI abstraction is particularly attractive in cloud settings. A tenant may submit a legacy in-memory analytics job written for a single large shared-memory server, while the cloud provider only has fragmented CPUs, memory, and devices across a CXL pod rather than one sufficiently large machine. Without SSI, the tenant must either wait for a large server to become available or rewrite the application into a distributed or CXL-aware design. With OS-level SSI, the provider can aggregate rack-scale resources across the pod and run the job transparently as if it were on one larger machine, without requiring any application changes.

Why not choose user-level or hypervisor-level SSI?

We consider three possible layers for realizing SSI on a CXL pod: (1) user-level SSI, i.e., distributed shared memory (DSM) libraries [15, 22, 41, 43, 47, 58], (2) hypervisor-level SSI [25, 51, 55], and (3) OS-level SSI [17, 24, 27, 44, 57, 62]. We choose OS-level SSI because it is the only approach that can *both* transparently run existing shared-memory applications and globally manage rack-scale resources.

User-level SSI, i.e., DSM libraries [15, 22, 41, 43, 47, 58], also lets programmers write as if on a single machine, but CXL-based DSMs still require *application-specific redesign* for efficient DRAM/CXL data placement. They also cannot provide the rack-scale resource pooling and transparent POSIX application execution that we target.

Hypervisor-level SSI [25, 51, 55] uses a distributed hypervisor to aggregate rack-scale resources. However, making applications actually work on a CXL pod still requires substantial guest-OS changes (e.g., in scheduling, memory placement, and system services). For SSI, implementing the system at the OS level is therefore more effective than doing so at the hypervisor level, because the OS has richer semantic visibility and more direct control over system policies.

3 State-Partitioned Microkernel

3.1 Challenges of building SSI atop CXL

Building OS-level SSI atop CXL requires (1) efficiency: utilizing shared memory for cross-machine synchronization, but should placing as minimized state as much on CXL, and (2) robustness: survive the OS under partial failures of physical machines, i.e., when a machine fails with partially updated shared state, shared state should not block or crash surviving machines that operate on it.

3.2 Why microkernels are a good fit

We find microkernel architecture a good fit for building SSI atop CXL for two main reasons:

(1) *Decoupled and Structured State Organization*: A microkernel gives system state a clearer organization than a monolithic kernel. At a coarse granularity, it decouples kernel state from service state: the kernel retains only a small amount of state for basic mechanisms such as IPC and scheduling, while most OS services, including file systems and device drivers, run in user space with their own state. As a result, major categories of system state are placed behind explicit boundaries, rather than being mixed inside the kernel.

At a finer granularity, many microkernels represent and organize state through capabilities [32], which represents system resources as objects with explicit relations among them [38, 53, 65, 68]. This gives system state a finer-grained and more explicit structure, making dependencies easier to reason about. By contrast, although monolithic kernels also maintain kernel objects, these objects are often spread across multiple subsystems and connected through implicit and intricate dependencies. Consequently, system state is harder to reason about, isolate, and partition correctly.

(2) *Easier service recovery*: microkernels can recover failed user-space system services by micro-rebooting them [23, 45]. For example, if a filesystem service fails, the system can restart that service without rebooting the whole OS. In contrast, in monolithic kernels, filesystems and many other services run inside the kernel and share kernel state with the rest of the system, making isolated restart much harder. Micro-rebooting is especially useful under partial failures, because if one machine fails and takes the service down with it, the service can be restarted on another machine without rebooting the whole OS.

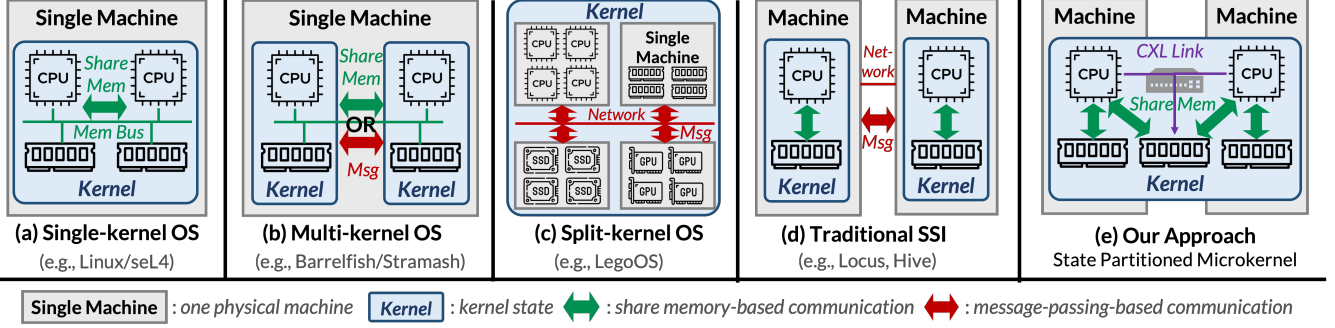


Figure 2: Kernel architecture of existing OSes and our proposed state-partitioned microkernel.

3.3 State-partitioned microkernel

Microkernel’s structured state abstraction and easier service recovery make it easier to achieve *efficiency* by placing as much state as possible on CXL, and *handle partial failures* of this minimized shared state while tolerating machine failures by micro-rebooting failed services.

Based on this, we propose a *state-partitioned microkernel architecture* for SSI atop CXL. It explicitly divides SSI state into three layers: (1) kernel-space modules, (2) user-space system services, and (3) applications. This structure lets us use layer-specific state-partitioning and partial-failure-handling mechanisms as summarized in Table 1.

3.4 Differences with Existing OSes

As shown in Figure 2, our proposed state-partitioned microkernel departs from existing OSes in three main aspects:

(1) *Cross-machine coordination with shared memory instead of message passing.* Single-machine OSes (Figure 2(a)) assume only local memory and do not support cross-machine coordination. Multi-kernel OSes [17, 18](Figure 2(b)), SSI OSes [24, 57, 62] (Figure 2(d)) and disaggregated OSes such as LegoOS [52] (Figure 2(c)) use multiple kernel instances and coordinate them via message passing. In contrast, StarfishOS directly constructs kernel-space data structures in shared memory for cross-machine coordination.

(2) *An unique partial-failure model.* Existing systems generally assume a single failure domain (for single-machine OSes, Figure 2(a) and (b)), or assume that when a machine fails, its CPU execution and OS state fail together (for SSIs and disaggregated OSes, Figure 2(d) and (c)). They all fail to handle the partial failure model introduced by CXL, where a machine’s CPU and memory may fail while its kernel and application state remains accessible in CXL. In contrast, StarfishOS is designed explicitly for this partial-failure model.

(3) *Extending resource disaggregation to CPUs.* Prior disaggregated systems mainly separate memory and storage while CPU execution remains organized per machine. For example, LegoOS [52] splits resources into CPU, memory, and SSD nodes, allowing multiple CPU nodes to share a common memory/storage pool, but each process still runs on a single CPU node. In contrast, StarfishOS extends disaggregation beyond memory and storage to CPUs across machines.

4 Design Overview

4.1 StarfishOS in Action: A MapReduce Case

We use a simple MapReduce program to illustrate the core capabilities and components required by StarfishOS. Running MapReduce across a CXL pod requires the following steps:

- *Step 1: Create a thread pool and place mapper/reducer threads across machines.* This requires StarfishOS’s **scheduler** to schedule threads on each CPU core in the pod.
- *Step 2: Initial threads load input data from the filesystem.* This requires (1) StarfishOS’s **filesystem services** to provide a global filesystem namespace so that threads across the CXL pod can read the input file regardless of where it is physically stored, and (2) **inter-machine IPC** to invoke the filesystem service.
- *Step 3: Mappers compute data and place intermediate results into shared memory buffers.* This requires StarfishOS’s **memory manager** to enable allocation of shared memory buffers from DRAM or CXL memory.
- *Step 4: After mappers complete, use conditional variables to wake up reducers.* This requires a **notification mechanism** that supports waking up threads on other machines.

4.2 Basic Components of StarfishOS

As shown in Figure 3, to support the above four steps StarfishOS is composed of the following components:

(1) *Bootloaders.* Each machine has a bootloader responsible for gathering its local resources, and coordinating with other bootloaders to boot StarfishOS. They build a unified kernel-space virtual address space, so that machines across the pod execute in a consistent address-space layout without pointer rebasing. Specifically, CXL is mapped at the same kernel virtual address on all machines, while each machine’s DRAM is mapped to a distinct address range.

(2) *Unified kernel-space modules* (§5) span multiple machines to provide core OS mechanisms, including IPC (*Step 2*), scheduling (*Step 1*), notification (*Step 4*), and memory management (*Step 3*). Together, these modules provide a unified kernel abstraction across CXL pod, and their state is distributed across per-machine DRAM and CXL.

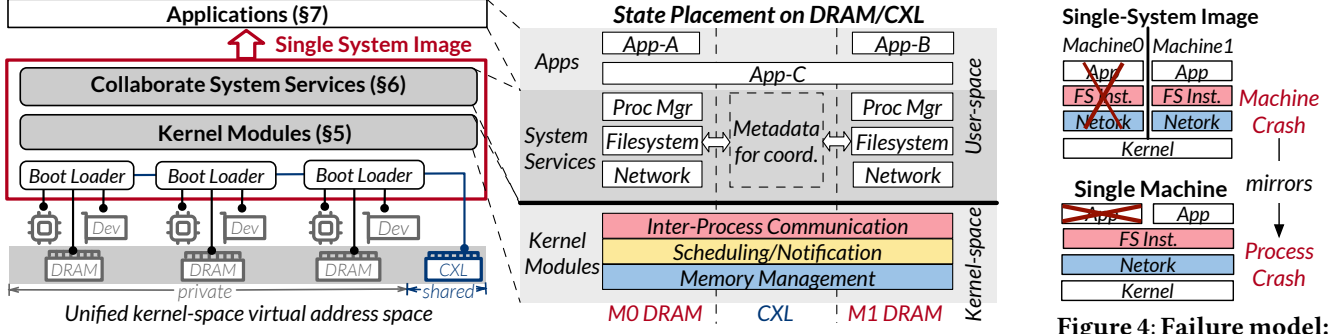


Figure 3: StarfishOS architecture. StarfishOS is an OS-level SSI with unified kernel-space modules (§5), collaborative user-space system services (§6), and bootloaders (§4). With StarfishOS, applications (§7) run on a CXL pod as if they were running on a single machine.

Table 1: State partition and partial failure handling of each component in StarfishOS.

	Component	Shared state	Partial failure handling
K-space func.	IPC (§5.1)	IPC queues & nodes	Detectably recoverable queues
	Notification (§5.3)	Waiting-lists & fake thread nodes	Detectably recoverable queues; libc timeouts mitigate higher-level primitives
	Scheduler (§5.2)	Inter-machine scheduling queues	Detectably recoverable queues
	Mem manager (§5.4)	CXL allocator’s metadata/data	Lock-free buddy system & light undo logs
U-space	Sys services (§6)	Global coordination metadata	Micro-reboot service instances on surviving machines and repair state gaps.
	Applications (§7)	CapGroup/VMspace/PMO/part-of-heap	Applications recover with their own mechanisms; the OS remains usable on surviving machines and reclaims resources from failed ones.

(3) *Collaborative system services* (§6) present a single global view across machines—a single PID space, filesystem namespace, and device namespace—so that applications can seamlessly use both local and remote services (for *Step 2*). We design multiple service instances running on different machines that operate collaboratively, keeping instance’s state in private memory while only coordination metadata in CXL.

(4) *Applications* (§7), with the above three components, can transparently run across the CXL pod.

4.3 Hardware, Failure Model, Base Microkernel

Hardware setup. We target a CXL pod hardware setup [63] (Figure 2 (e)), where rack-scale machines are equipped with *private memory* (i.e., local DRAM) and *shared memory* (i.e., memory connected through the CXL switch [3]).

We assume hardware cache coherence is available for CXL, but keep its usage as small as possible (several hundred MiBs). Although prior work [35] and vendors [37] suggest supporting coherence only for a small region, this mainly reflects hardware-complexity tradeoffs rather than a limitation: both Arm’s discussion [46] and FPGA prototypes [60] indicate that broader coherence is feasible. Still, StarfishOS keeps coherent-memory usage as small as possible to avoid coherence maintenance overhead [60].

We assume CPU caches are lost on machine failure, so StarfishOS relies on cacheline flushes for partial-failure handling. Hardware support such as global persistent flush [50] (an eADR-like [36] mechanism for persisting cache state

to CXL after failure) could eliminate cacheline flushes and further improve performance.

Failure model. StarfishOS tolerates partial failures [69]. It assumes that CXL memory is always reliable and available as prior works [66, 69], while physical machines may fail independently. StarfishOS assumes fail-stop failures of entire physical machines (including its CPU and DRAM), but not fail-slow or Byzantine failures. StarfishOS also assumes a reliable global detector for machine failures.

Behavior under partial failures. Under failures of one or several machines, StarfishOS preserves OS functionality on surviving machines, but does not provide transparent application recovery. (1) Preserving OS functionality means that all kernel-space mechanisms and system services remain available on surviving machines: surviving processes can continue invoking IPC, scheduling, and memory-management APIs, and system services continue serving requests without losing state. (2) No transparent application recovery means that applications running on failed machines terminate abruptly, analogous to process crashes in a conventional single-machine OS, while applications on surviving machines continue unaffected (as shown in Figure 4).

Base microkernel and our modifications. StarfishOS is built on ChCore [13, 31, 65], which is a capability-based microkernel like seL4 [38]. ChCore provides IPC, scheduler/notification, and memory management in kernel space, and system services such as process management and filesystems in user space. ChCore uses musl-libc [8] for POSIX

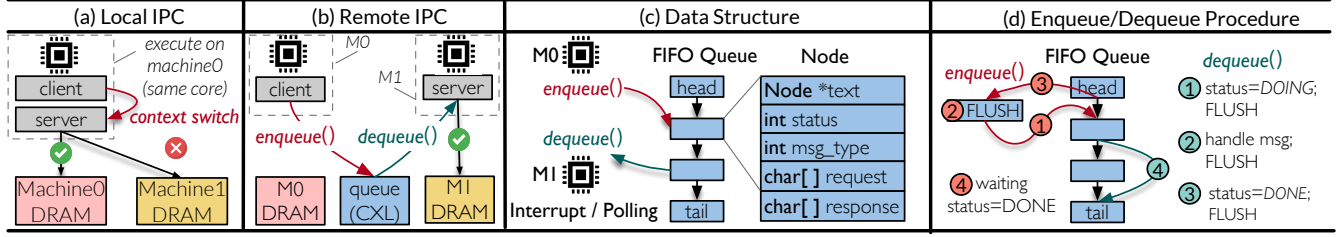


Figure 5: Inter-machine IPC. (a) Local IPC follows the L4/seL4 style: to invoke IPC to a system service, the client thread context switches to the server thread (to execute server’s code, e.g., to read a file). However, cross-machine IPC can not use this design since after context switch, the thread still run on machine 0, and can not access remote server’s state. (b) StarfishOS therefore adopts a message-passing design for cross-machine IPC, each server has a FIFO queue for handling remote requests.

compliance, and can run various applications (including Redis [9], Memcached [7], RocksDB [10], etc.). Although our prototype is built on a specific microkernel, the techniques described here also apply to other microkernels.

5 Kernel-Space Modules

This section describes how we realize SSI by extending kernel-space modules to support cross-machine functionalities.

Our *core design principle* is to partition each kernel module’s state into DRAM and CXL data structures for intra- and inter-machine mechanism (§5.1–§5.4), while making CXL data structures partial-failure resilient using lock-free, detectably recoverable data structures [16, 29] (§5.5). Table 1 summarizes our used methods for each component.

5.1 Inter-Process Communication

IPC is the microkernel’s mechanism for invoking system services. In our base kernel, IPC follows the L4/seL4 style [1, 19]: it is an function call-like, user-controlled context switch to a server thread without scheduler involvement. This design works for intra-machine local IPC, but it does not extend naturally to cross-machine remote IPC when the server’s address space and state remain in remote DRAM. We therefore keep the original design for local IPC, and implement cross-machine IPC as message passing over CXL.

Local IPC does not extend across machines (Figure 5 (a)). In our base kernel, local IPC follows the L4/seL4 style: the client transfers control directly to the server thread, which then executes in the server’s address space. For a remote server, however, its address space and state remain on the remote machine, so switching control alone is not sufficient. A straightforward alternative is to place server state in CXL so that multiple machines can directly operate on it. We do not choose this design because: (1) it would consume a large amount of scarce cache-coherent CXL memory; (2) under partial failure, a machine crash may expose partially updated server’s state to other machines, making partial failures harder to recover from.

Remote IPC (Figure 5 (b)). We therefore adopt a CXL-based message-passing design for cross-machine IPC. As shown in Figure 5 (b), each service provides a dedicated handler thread for cross-machine IPC, separate from its local

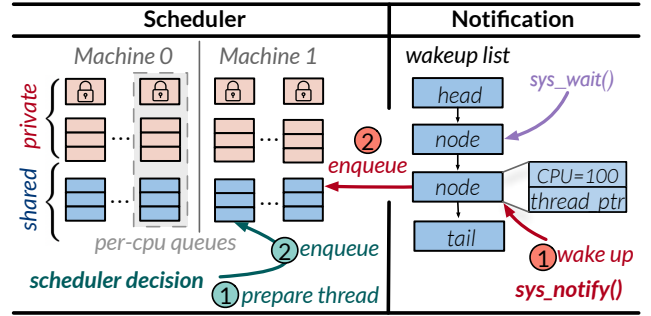


Figure 6: Scheduler/Notification.

IPC. Each such handler thread owns a FIFO request queue in CXL for cross-machine IPC requests. The client creates a message node on CXL (Node), prepares the request, enqueues it to the target machine’s queue, and, if needed, waits for `status==DONE`; the server dequeues the node, handles the request, writes the response, and sets the status to DONE.

5.2 Scheduler

StarfishOS extends thread scheduling beyond a single machine: it can assign a thread to any processing core in the CXL pod, including remote cores. Once assigned, the scheduler on the target core treats the thread as a local schedulable entity and schedules it according to the scheduling policy.

Local scheduling. As shown in Figure 6, the base microkernel uses per-core scheduling queues (*private queue*) that store runnable *thread control blocks*. A thread control block contains the execution state needed to resume a thread (e.g., saved registers), and the scheduling state (e.g., priority, affinity). Scheduler on each core selects a runnable thread from its private queue and context-switches to it.

Remote scheduling. To support scheduling a thread on remote machines, StarfishOS equips each core with an additional *shared queue* in CXL (Figure 6). Threads assigned to a remote core are enqueued into shared queue, allowing the target core to admit them into its local queue for scheduling. The shared queue is checked along the target core’s normal scheduling path, thus avoiding the overhead of sending explicit cross-machine scheduling requests. Note that a thread scheduled to a remote machine must already reside either in that machine’s private memory (for notifications, §5.3) or in

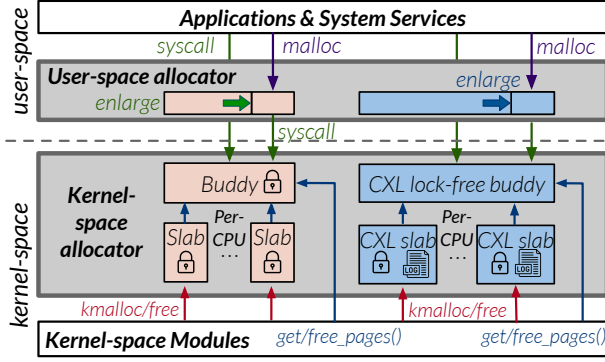


Figure 7: Memory Manager.

CXL (for cross-machine application execution, §7), so that the remote machine can access it.

5.3 Notification

Notification is the kernel mechanism for event-driven wakeup and serves as the basis for higher-level synchronization primitives such as timeouts, futexes, and semaphores. StarfishOS extends notification to support cross-machine wakeup.

Local notification. As shown in Figure 6, our base microkernel maintains a *waiting-list* of threads waiting on a notification. To wait for an event, an application thread invokes `sys_wait` to insert its *thread control block* into the waiting-list. When the event occurs (i.e., `sys_notify` is invoked or a timeout fires), the kernel wakes one waiting thread by dequeuing it and scheduling it.

Remote notification. StarfishOS supports cross-machine notification by placing the notification’s waiting-list in CXL. Thus, all machines can invoke `sys_wait/sys_notify` that operate on the shared waiting-list. However, the waking machine may still be unable to directly access the target thread’s control block in another machine’s private DRAM. To address this, each waiting-list node stores a pointer to the target thread together with its `cpu_id`, allowing the waiting machine to schedule the target thread.

5.4 Memory Management (Kernel- and User-space)

Memory management in StarfishOS support both machine-local DRAM and pod-wide CXL memory.

Original memory management. As shown in the left part of Figure 7, similar to conventional kernels, the base microkernel uses a buddy/slab organization [20, 39] in kernel space and a libc allocator in user space. The kernel allocator serves kernel-space module’s allocations, while the user-space allocator obtains memory from the kernel and serves user-space processes’ allocations.

StarfishOS’s memory management. StarfishOS extends memory management to support both machine-local DRAM and pod-wide CXL memory. It uses a separate DRAM allocator on each machine and a shared CXL allocator for the entire pod. In kernel space, StarfishOS retains the conventional buddy/slab structure, but replaces the CXL buddy allocator with a lock-free buddy system [64] for partial failure resilience

(discussed later in §5.5). The slab allocator remains per-CPU: allocation is served by the local slab instance, while deallocation is routed back to the owning machine for reclamation. In user space, StarfishOS extends the libc allocator to manage both DRAM and CXL memory; applications can choose the target memory type, and the default is the DRAM of the machine on which the application runs.

5.5 Partial Failure Handling

Since StarfishOS places shared kernel state on CXL, it must ensure that partial failures neither corrupt shared state nor block surviving machines. For example, if a machine fails while holding a lock in CXL, the lock will not be released and may block other machines.

To avoid this, our *design principle* is to use lock-free and crash-consistent data structures for shared kernel state [16, 29]. Lock-freedom prevents failed machines from blocking others, while crash-consistency ensures that crashes do not leave shared state corrupted and that recovery can handle in-flight operations [16, 29].

We defer details of our implementation to supplementary material due to space constraints.

Durable Queue for IPC, Scheduler, Notification. As listed in Table 1, we implement the IPC queue, scheduler queue, and notification waiting-list with DurableQueue, which is a lock-free and detectably recoverable FIFO queue proposed by Friedman et al. [29]. As shown in Figure 5 (d), enqueue and dequeue linearize with a single CAS, so a partial failure never leaves the queue itself corrupted or permanently blocked. Building on this property, StarfishOS handles failures differently depending on whether they occur at the enqueuer or dequeuer side:

- *Enqueuer Crash:* If the enqueuer’s machine fails during enqueue, the queue remains usable because the operation is lock-free. Other machines can still enqueue IPC messages, schedule threads, or wait for events.
- *Dequeuer Crash:* If the dequeuer’s machine fails during dequeue, the operation may be lost or only partially executed, but it does not block other machines. Recovery is component-specific: IPC returns ABORT or relies on service-level retry; scheduler reconstructs local state and retries; notification returns ABORT, allowing libc to report a timeout for `sys_wait`.

CXL allocator. StarfishOS maintains kernel-space buddy/slab allocator and user-space libc allocator for CXL memory. We ensure their partial failure resilience as follows:

- *Lock-free Buddy:* We choose LLFree [64], a lock-free and crash-consistent allocator, to implement CXL buddy system. Under partial failure, the buddy state remains consistent, and other machines can still allocate/free memory without blocking since LLFree is lock-free.
- *Per-CPU Slab with Undo-log:* Each CPU has its own slab instance, so allocation on slab stays local and does not block other machines. Free operations on slab allocator

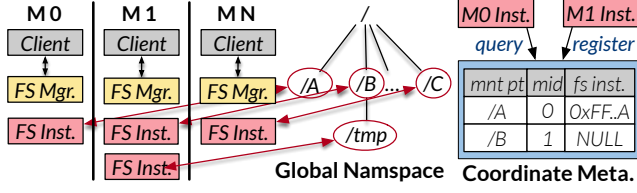


Figure 8: Collaborative filesystem service. Filesystem instances are distributed across machines, each owns a disjoint part of file namespace.

are forwarded asynchronously to the owner machine. To safely reuse slab instances after recovery, StarfishOS keeps a lightweight in-memory undo-log to undo in-flight slab operations.

- *Libc allocator*: each application uses an independent libc allocator instance. Thus, the failure of libc allocator will not block other applications.

6 Collaborative System Services

System services in StarfishOS are distributed as collaborative user-space instances. This keeps most service state in private DRAM, leaves only ownership metadata in CXL. This section describes service-state partitioning (§6.1) and partial-failure recovery (§6.2).

6.1 State Partitioning with Collaborative Instances

StarfishOS partitions each system service into multiple owner instances instead of building one shared service for the whole pod. A single shared instance would push most service state into scarce cache-coherent CXL memory and make partial failures harder to contain: a machine crash during a service update could leave shared metadata in an inconsistent state that affects every client.

StarfishOS therefore keeps only ownership metadata in CXL and leaves service state inside per-machine DRAM. Each instance runs on one machine and owns a disjoint part of the global resource space, while the CXL metadata only records how to find that owner, e.g., mount-point-to-filesystem-instance mappings and PID-space ownership metadata. Clients use this metadata to route service requests with local or cross-machine IPC, without sharing the instance’s full internal state.

The filesystem in Figure 8 is the concrete example. Each machine runs a filesystem manager and several filesystem instances, and each instance owns one mount point. On the first access to a path, the client asks the manager which instance owns the mount point and caches the result; later requests go directly to that instance. If the owner is remote, the request uses cross-machine IPC. Other system services follow the same pattern.

6.2 Partial Failure Handling with Micro-rebooting

StarfishOS aims to preserve system-service availability for surviving applications under partial machine failures, including network communication, complete filesystem access, and

Table 2: Objects managed by StarfishOS.

Object	Description
CapGroup	capability array that references other objects
Thread	execution context and scheduling state
VMspace	virtual address-space metadata
PMO	physical memory pages
IPC	inter-process communication state
Notification	synchronization object

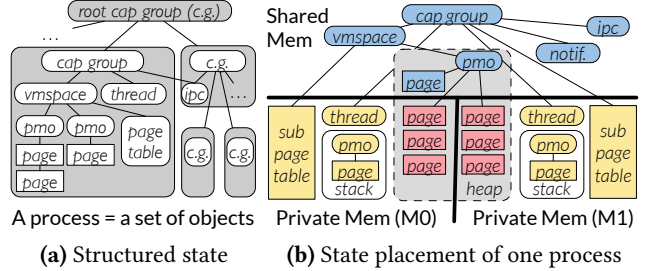


Figure 9: Application state partition.

process management. Partial failures are handled by recovering only the lost service instances. Because system services are user-space processes, StarfishOS can micro-reboot a failed instance, i.e., restart just that service instance on a surviving machine without restarting the whole pod [45].

Whether recovery is needed depends on the service type. Uniform services, such as networking, provide the full interface at every instance, so clients can reconnect to any surviving instance after a failure. Process management is handled similarly in our design: new process or thread creation can be redirected to a surviving manager, while cleanup work for resources owned by the failed manager can be delayed or delegated. Non-uniform services that own disjoint resources, in contrast, require explicit instance recovery.

In our current design, filesystem is the only non-uniform service that requires explicit instance recovery. After a machine failure, StarfishOS starts a replacement filesystem instance on another machine and rebuilds its state with Ananke-style p-log replay [45]. The p-log is stored in shared memory and records each mutating filesystem operation before the service replies, so any update already reported as complete to the client is preserved even if it has not yet reached SSD. The replacement instance replays the p-log and resumes serving the same mount point. Clients can then reconnect to the recovered instance and send requests.

7 Cross-machine Applications

With the kernel modules and collaborative system services in place, applications can already run transparently on a CXL pod. The remaining question is where their state should live. StarfishOS therefore partitions application state for efficiency and defines how applications recover from partial failures.

7.1 Application State Partition

State partitioning follows one core principle: only cross-machine state should pay CXL latency. Per-thread and machine-local state stays in DRAM, while state that must remain visible across machines goes to CXL. This keeps the common case local without breaking cross-machine execution.

Structured state organization. Capability-based micro-kernels [38, 53, 65, 68] represent system state as typed objects, and use capabilities to record which objects a process can reference and use. As shown in Figure 9(a) and Table 2, a process is therefore organized as objects such as *CapGroup*, *Thread*, *VMspace*, and *PMO*. *CapGroup* stores the capabilities to these objects, making inter-object reachability explicit. This object-based organization gives StarfishOS a concrete way to track, place, and migrate application state at object granularity instead of treating the process as a black box.

Where to place each object? StarfishOS uses OS semantics to choose object placement. Per-thread state stays private, globally visible control state goes to CXL, and heap pages are shared only when accesses prove they must be shared. Figure 9(b) summarizes these choices.

(1) *Private state stays in DRAM.* Thread context and stacks are per-thread, so keeping them private avoids adding CXL latency. Each machine also keeps the local page-table state needed for its own execution path (further discusses in §7.2).

(2) *Shared control state lives in CXL.* *CapGroup*, *VMspace*, *PMO* metadata, *IPC*, and *Notification* state must remain visible across machines, so StarfishOS places them in shared memory. These objects are small or updated infrequently enough that the added CXL cost is acceptable.

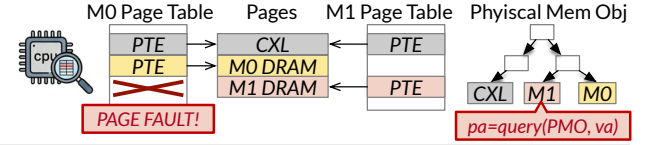
(3) *Heap pages are shared on demand.* The heap mixes local and cross-machine data, so placing the whole heap in CXL would waste bandwidth. StarfishOS therefore uses page-fault-driven migration to keep heap pages in DRAM by default and move only remotely accessed pages to CXL.

7.2 Page-fault-driven migration for Heap

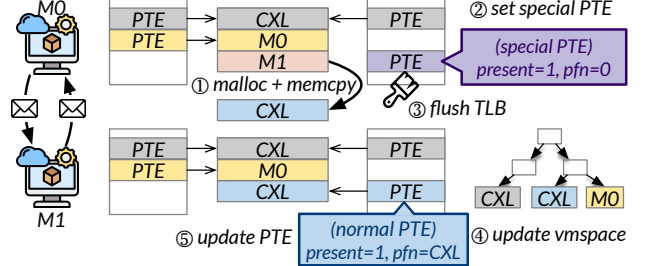
Page-fault-driven migration enables transparent lazy sharing of heap pages. A page is allocated in private DRAM of the machine that first touches it. When a remote access faults, StarfishOS migrates the page to CXL and makes it shared. Each machine keeps a sub page table, so a virtual page may be mapped on one machine and absent on others until sharing is required. Figure 10 shows the migration path when machine0 accesses a page currently owned by machine1.

1. Machine0 faults on a virtual address and queries *VMspace* to determine whether the address has a valid mapping and, if so, whether the mapping points to a DRAM page. If both conditions are met, machine0 identifies the owner machine and sends that machine a migration request through the mechanism in §5.1.
2. Machine1 performs the migration. request contains *vmSPACE* and *va*. It allocates a CXL page and copies the data from

1. M0 page fault & Page exists and is on local DRAM



2. M0 Send Msg to ask M1 to migrate page



3. M0 receive ACK and map page

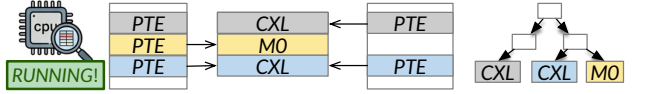


Figure 10: Transparent page placement across DRAM and CXL with Page-fault-driven migration.

DRAM to CXL (①). It then atomically replaces the original PTE with a special migration entry (②) so concurrent accesses block until the migration finishes. After flushing stale TLB entries (③), machine1 updates *VMspace* (④) and installs a new PTE that points to the CXL page (⑤). Concurrent accesses waiting on the migration entry will then observe the new PTE and refill the correct TLB entry.

3. After receiving the ACK, machine0 installs its local mapping to the CXL page and resumes execution.

When CXL becomes full, StarfishOS migrates pages back to DRAM and invalidates stale remote mappings. The current policy first reclaims older migrated pages and prefers pages shared by fewer machines.

7.3 Handling Partial Failure of Applications

StarfishOS does not hide partial failures from applications; it relies on applications' own recovery mechanisms. Transparent system-wide rollback would add cost in the common case, and many applications already rely on their own recovery mechanisms such as WAL, restart, or replay from saved intermediate state.

StarfishOS instead preserves the execution environment that survives the failure. Surviving machines remain usable, files and flushed CXL state remain accessible, and applications can either restart from their own recovery data or continue with remaining threads if their state tolerates partial loss. §8.6 evaluates both recovery styles.

Resource reclamation remains automatic. StarfishOS traverses the capability tree to reclaim objects from exited or

Table 3: DRAM/CXL latency/bandwidth on our platform and reported FPGA/ASIC results. GB/s/channel for DRAM, GB/s/line for CXL. FPGA performance is based on real FPGA prototype [60], and ASIC performance is based on reports from chip vendors and IP providers [28].

Metric	Local	Remote	Local	Remote	FPGA	ASIC
	DRAM	DRAM	CXL	CXL	CXL	CXL
Latency (ns)	107	160	241	383	456	170
Bandwidth (GB/s)	52	30	2	0.3	32	64

failed processes, and CXL-resident cap groups let the recycler find the resources of cross-machine applications even after one machine has failed.

8 Evaluation

We evaluate StarfishOS using microbenchmarks and real applications to answer the following questions:

- How do the basic modules of StarfishOS perform? (§8.2)
- How does partitioned state contribute to efficiency? (§8.3)
- How can applications benefit from StarfishOS in auto-scaling and resource utilization use cases? (§8.4 and §8.5)
- How can applications recover from failures? (§8.6)

8.1 Experimental Setup

Testbed. Our evaluation platform has four Intel® Xeon® Gold 6418H CPUs (Sapphire Rapids), each with 24 physical cores and 64 GiB of DRAM. The server also includes a real 128 GiB CXL 1.1 memory device. CPU frequency is fixed at 2.1 GHz with Hyper-threading disabled, Intel® Turbo Boost disabled and NUMA balancing disabled.

Environment. Due to the lack of real CXL switch hardware, we emulate an 8-machine CXL pod by running 8 QE-MU/KVM instances, each with 12 physical CPU cores. Every two emulated machines share a NUMA node, and all machines access a CXL memory device. Except for applications running on StarfishOS, all other experiments are conducted on Linux v5.15 with Ubuntu 22.04. Each experiment is performed 3 times following an initial warm-up run. VM exits account for 3–5% of runtime in our evaluated applications, mainly due to EPT_violation and CPU_HLT.

As shown in Table 3, compared to CXL 3.0, our CXL 1.1 environment has comparable latency (246/384 ns v.s. 456 ns in FPGA and 170 ns in ASIC), while the bandwidth is substantially lower (2/0.3 GiB/s v.s. 32/64 GiB/s/line in FPGA and ASIC). Besides, our platform does not model the cacheline invalidation overhead in CXL 3.0 [60], which increases cache miss latency by 2–3× [60]. Nevertheless, these difference do not affect the conclusions of our evaluation: the lower bandwidth only put our system at a disadvantage, and the latency is comparable; moreover, our partitioning strategy keeps most accesses on local DRAM and limits CXL accesses to a small fraction of the workload, thereby reducing the impact of CXL 3.0 overheads.

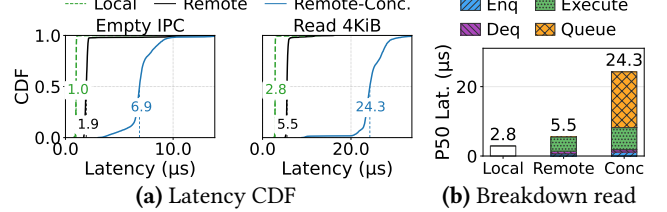


Figure 11: IPC latency CDF and breakdown.

Workloads. We evaluate StarfishOS using microbenchmarks and the following real-world applications. All applications running on StarfishOS has no code changes.

- *Phoenix-2.0* [49]: single-machine mapreduce application.
- *DBx1000* [5, 67]: a single-node OLTP database system designed to make DBMS scalable on 1000-core processors.
- *GeminiGraph* [72]: a graph processing application with both single-machine and distributed modes.
- *CNN* [2]: a single-machine neural network inference.
- *Redis* [9], *Memcached* [7] and *LevelDB* [6]: memory-based and disk-based key-value stores.

8.2 Microbenchmarks

This section presents performance of basic kernel modules.

IPC. We evaluate local and remote IPC latency using two workloads: (1) *empty*, an IPC call with no server-side execute, and (2) *read 4 KiB*, which reads a 4 KiB file through IPC to filesystem. For remote IPC, we measure both the single-sender case (labeled as *Remote*) and four concurrent senders case (labeled as *Remote-Conc.*); local IPC is labeled as *Local*.

Figure 11(a) shows the latency CDF of local and remote IPC. For *empty*, P50 latency of local IPC is about 1.0 μs, while remote IPC is about 1.9 μs without concurrent senders and 6.9 μs with concurrent senders. For *read 4 KiB*, local IPC latency increases to 2.8 μs as the request includes executing filesystem read, and remote IPC latency increases to 5.5 μs and 24 μs without and with concurrent senders.

Figure 11(b) shows the latency breakdown of *read 4 KiB*; the *Local* column is a white bar marking P50 total only (component breakdown is given in text below), while *Remote* and *concurrent senders* use the stacked breakdown. For remote IPC, enqueue and dequeue on the IPC queue takes about 1.4 μs in total, and handling the request takes about 2.0 μs. For remote IPC with concurrent senders, the extra 19 μs overhead is caused by queuing delay. For local read IPC (in total 2.8 μs), besides about 1.8 μs of executing the filesystem read, the time is spent on context switching (706 ns), IPC logic (200 ns), and kernel trap overhead (130 ns).

Scheduling/Notification. We evaluate the end-to-end latency of local and remote scheduling and notification. For scheduling, we start from enqueue a thread, to when the thread is actually scheduled. For notification, we start from calling *sys_wait* to when the thread is actually notified. We found that remote scheduling/notification adds negligible overhead compared to local scheduling/notification. The

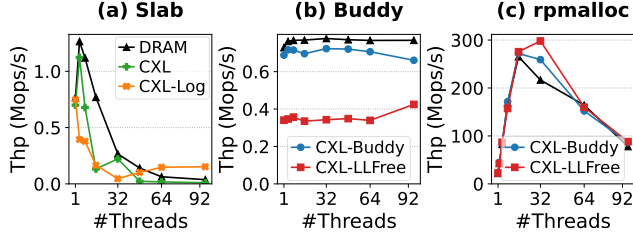


Figure 12: Memory allocator throughput.

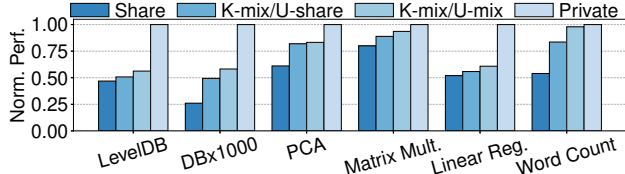


Figure 13: Performance normalized to *Private* (throughput ratio for LevelDB/DBx1000; private execution time over measured time for others) across state-partitioning choices.

only overhead is extra enqueue/dequeue, adds about 1–2 μ s latency to ms-level scheduling/notification.

Memory Allocation. We evaluate throughput of kernel-space slab/buddy allocator and the user-space libc allocator, each backed by either DRAM or CXL. (1) *Slab* uses kernel-space *kmalloc* to allocate/free 64–1024 B objects following the *rpmalloc* benchmark [11]; We compare slab allocators backed by *DRAM*, *CXL*, and *CXL-Log*, which incorporates our undo-log mechanism. (2) *Buddy* uses kernel-space *get/free_pages* to randomly allocate and free 4 KiB and 2 MiB pages with equal probability. We compare buddy allocators backed by *DRAM*, *CXL-Buddy* (buddy allocator implemented with original lock-based buddy system), and *CXL-LLFree* (our lock-free buddy allocator based on *LLFree* [64]). (3) *rpmalloc*, i.e., our libc allocator, allocates random-sized user-space objects in the same range as *Slab*. We compare libc allocators backed by *DRAM* and *CXL*.

Figure 12 shows that kernel-space allocation is noticeably more sensitive to CXL placement than user-space allocation. In contrast, the impact on user-space *rpmalloc* is smaller, because most *malloc/free* operations are served from thread-local state (e.g., free lists), and therefore touch cached local metadata more often than the kernel allocator. Also, our usage of *Undo-log* mechanism and *CXL-LLFree* decreases kernel-space allocator’s performance due to cacheline flush, but have minimal impact on user-space *rpmalloc*.

8.3 Effect of State Partitioning

We evaluate the effectiveness of StarfishOS’s state partitioning by running several applications across two machines with four memory configurations: (1) *Shared*: place all kernel and application state in CXL; (2) *K-mix/U-share*: partition only kernel state, while all application state in CXL; (3) *K-mix/U-mix*: partition both application and kernel state; (4) *Private*: all state in private DRAM (ideal case).

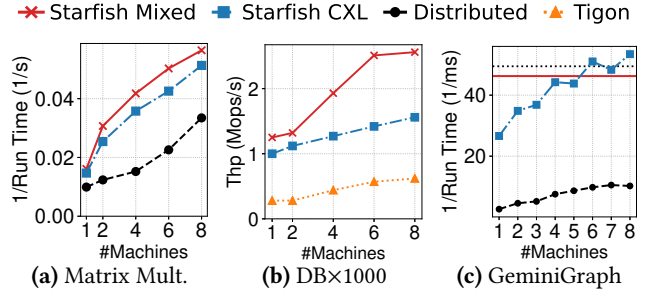


Figure 14: Auto-scaling single-machine applications.

Table 4: Mem usage in auto-scaling apps. We fix DBx1000 and Tigon [35]’s CXL usage to 500 MiB.

Mem	Matrix Mult.	DBx1000	Tigon	GeminiGraph
Total	1 GiB	30 GiB	30 GiB	50 GiB
CXL	109 MiB	500 MiB	500 MiB	600 MiB

Figure 13 shows that StarfishOS’s state partitioning (*K-mix/U-mix*) achieves 1.2–2.2 $\times$ performance improvement compared to placing all state in shared memory (*Share*), and 1.0–1.7 $\times$ improvement over only partitioning kernel state (*K-mix/U-share*). This presents the benefit of fine-grained state partitioning. CPU-intensive applications (e.g., PCA, Matrix Multi.) achieve similar performance with the *Share* configuration because most data can be cached in CPU cache.

8.4 Auto-scaling Applications

This case study evaluates whether StarfishOS can auto-scale existing single-machine applications, and outperform their distributed and CXL-optimized versions.

Configs. We use three unmodified multi-threaded applications: (1) CPU-intensive MapReduce, i.e., Matrix Multiplication in Phoenix [49] (multiply one 4000 $\times$ 4000 matrix); (2) memory-intensive database, i.e., DBx1000 [5, 67] running TPCC [12] with 64 warehouses, 50% New-order, 50% Payment, 15% cross-warehouse transactions; and (3) graph processing application, i.e., GeminiGraph [72] running PageRank [48] on twitter-2010 [40]. In *StarfishOS-CXL*, all heap and page-table state is placed in CXL. In *StarfishOS-Mixed*, only the shared portion of the heap is placed in CXL using page-fault-driven migration (§7.2), and each machine keeps a private page table in DRAM. We also compare applications running on StarfishOS against their distributed baselines running on Linux. For Phoenix, we implement a distributed version that uses network message passing for intermediate data. For DBx1000, we use open-source Tigon [35], a database optimized for CXL. For GeminiGraph, which has both single-machine and distributed versions, we use its native distributed MPI implementation.

Results. Figure 14 shows that applications can scale from 1 to 8 machines with 3.5 $\times$ (Matrix Mult.), 2.0 $\times$ (DBx1000), and 2.0 $\times$ (GeminiGraph) speedup. Across all three applications, *StarfishOS-Mixed* consistently outperforms *StarfishOS-CXL*, indicating the effectiveness of placing only shared

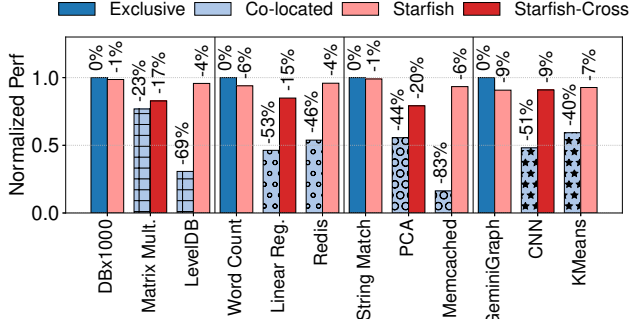


Figure 15: Overall performance of 8-machine cluster w/ and w/o StarfishOS’s cross-machine execution capability. Bars with the same hatch pattern are co-located on the same machine.

part of the heap in CXL. Besides, applications running on StarfishOS achieves $4.2\times$ (GeminiGraph) and $1.7\times$ (Matrix Mult.) speedup compared to their distributed versions. And DBx1000 running on StarfishOS outperforms database optimized for CXL, i.e., Tigon [35] by $2\times$. Besides, StarfishOS only consumes several hundred MiB of CXL memory, which is less than 1% of application’s total memory usage.

Benefits of StarfishOS over distributed versions. The main benefit of StarfishOS is that it replaces software-managed distributed communication with direct accesses to shared CXL-resident state. Instead of forcing applications to explicitly partition data and exchange messages, StarfishOS migrates only the state that becomes shared, so threads on different machines can coordinate through ordinary load/store accesses to the same memory region. This benefit appears differently across workloads. For Matrix Mult., StarfishOS places shuffle-related intermediate results in CXL, so mapper and reducer threads on different machines can share them without network copies. For GeminiGraph, frequently shared graph state can be placed in CXL, which reduces both up-front partitioning constraints and the amount of message exchange needed to propagate updates.

For DBx1000, we compare against Tigon’s open-source implementation, which is redesigned to place shared state on CXL and eliminate the message-passing overhead of 2PC. StarfishOS still outperforms Tigon by $2\times$. This is not a strict apples-to-apples comparison, since Tigon is an application-specific redesign while StarfishOS runs unmodified DBx1000. We include it to show that transparent OS-level support can remain competitive even against a hand-engineered CXL-aware design.

8.5 CPU Resource Utilization

StarfishOS allows applications to run across multiple machines to make use of stranded CPU resources. In Figure 15, we evaluate 12 popular datacenter applications, divided into four groups. Each group contains three applications, which are run on two machines. Traditional OSes (e.g., our base microkernel) restrict each application to a single machine,

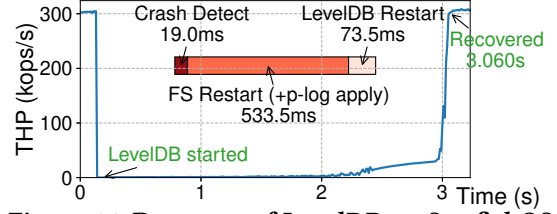


Figure 16: Recovery of LevelDB on StarfishOS.

so in each group, one application runs exclusively on one machine (*Exclusive*), while the remaining two applications are co-located on the other machine (*Co-located*). StarfishOS enables an application to run across machines with no code change. Therefore, in each group, two applications execute on single machines (*StarfishOS*), and the remaining application runs across the two machines (*StarfishOS-Cross*).

All results in Figure 15 are normalized to the performance of exclusively using 12 cores on our base microkernel. *Co-located* applications running on a traditional OS achieve only 17%–77% of the performance compared to exclusive execution. By contrast, cross-machine applications running on StarfishOS (*StarfishOS-Cross*) achieve 83%–93% of the performance of exclusive execution, which highlights the benefits of StarfishOS’s ability to utilize stranded CPU resources. Notably, we evaluated several other grouping and co-location plans, and all of them show similar benefits of StarfishOS.

8.6 Partial Failure Resilience

To evaluate how applications recover from partial failures of machines, we deploy LevelDB on Machine-A and crash Machine-A. Machine failure is simulated by killing the QEMU instance. After 19 ms, the failure detector detects the failure of Machine-A, and micro-reboots an empty filesystem instance on another machine in 5.5 ms. The empty filesystem instance recovers state gap with the killed instance following Ananake [45]. Specifically, it replays the p-log in 528 ms (to replay 446 entries after last LevelDB’s flush). LevelDB can then reconstruct its in-memory state in 73 ms and warm up its execution in 3 s.

9 Related Work and Future Directions

Related work. StarfishOS is inspired by lot of prior works, but combines them in a different setting. Our cross-machine kernel modules and collaborative services build on decades of SSI design [17, 24, 44, 57, 62]. Our state partitioning draws inspiration from SASOS [26, 33, 42], Corey [21], Mitosis [14], which highlights explicit state management benefits. For partial-failure resilience, StarfishOS adopts lock-free, detectably recoverable data structures [16, 29] and lock-free allocators [64] motivated by recent works on CXL partial-failure resilience [61, 69, 73]. Our page-fault-triggered migration and replicated page-table design are also inspired by prior NUMA and page-table replication work [14]. Our contribution is not any one of these techniques in isolation, but integrating them into an OS-level SSI that transparently

runs unmodified applications on a CXL pod while preserving availability under partial failures.

Future Directions. CXL-based SSI is a promising abstraction for distributed computing, and several opportunities remain to improve StarfishOS, including further optimization of kernel mechanisms such as batched remote IPC, better system services such as a shared CXL-backed filesystem with fault tolerance, and more effective pod-wide scheduling.

10 Conclusion

StarfishOS is the first OS-level SSI to use CXL’s shared-memory primitive for efficient cross-machine synchronization. Built on a state-partitioned microkernel, it keeps CXL-resident state small, provides OS-level partial-failure resilience, and lets single-machine applications scale transparently across multiple machines with substantial speedups and modest overhead.

References

- [1] 2019. How to and how not to use seL4 IPC. <https://microkerneldude.org/2019/03/07/how-to-and-how-not-to-use-sel4-ipc/>.
- [2] 2025. CNN. <https://github.com/ekut-es/pico-cnn>.
- [3] 2025. Compute Express Link (CXL). <https://www.computeexpresslink.org/>.
- [4] 2025. CXL Consortium. Compute Express Link Specification, Revision 3.1, 2025. <https://computeexpresslink.org/wp-content/uploads/2025/02/CXL-3.1-Specification.pdf>.
- [5] 2025. DBx1000. <https://github.com/yxymit/DBx1000>.
- [6] 2025. LevelDB. <https://dbdb.io/db/leveldb>.
- [7] 2025. Memcached - a distributed memory object caching system. <https://memcached.org/>.
- [8] 2025. Musl Libc. <https://musl.libc.org/>.
- [9] 2025. Redis. <https://redis.io/>.
- [10] 2025. RocksDB Home Page. <https://www.rocksdb.org>.
- [11] 2025. Rpmalloc Benchmark. <https://github.com/mjansson/rpmalloc-benchmark>.
- [12] 2025. TPC Benchmark C. <https://www.tpc.org/tpcc/>.
- [13] 2025. TreesLS. <https://ipads.se.sjtu.edu.cn/projects/treesls.html>.
- [14] Reto Achermann, Ashish Panwar, Abhishek Bhattacharjee, Timothy Roscoe, and Jayneel Gandhi. 2020. Mitosis: Transparently Self-Replicating Page-Tables for Large-Memory Machines. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS ’20). Association for Computing Machinery, New York, NY, USA, 283–300. <https://doi.org/10.1145/3373376.3378468>
- [15] C. Amza, A.L. Cox, S. Dwarkadas, P. Keleher, Honghui Lu, R. Rajamony, Weimin Yu, and W. Zwaenepoel. [n. d.]. TreadMarks: Shared Memory Computing on Networks of Workstations. 29, 2 ([n. d.]), 18–28. <https://doi.org/10.1109/2.485843>
- [16] Hagit Attiya, Ohad Ben-Baruch, Panagiota Fatourou, Danny Hendler, and Eleftherios Kosmas. 2022. Detectable recovery of lock-free data structures. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Seoul, Republic of Korea) (PPoPP ’22). Association for Computing Machinery, New York, NY, USA, 262–277. <https://doi.org/10.1145/3503221.3508444>
- [17] Antonio Barbalace, Marina Sadini, Saif Ansary, Christopher Jelesnianski, Akshay Ravichandran, Cagil Kendir, Alastair Murray, and Binoy Ravindran. 2015. Popcorn: bridging the programmability gap in heterogeneous-ISA platforms. In *Proceedings of the Tenth European Conference on Computer Systems* (Bordeaux, France) (EuroSys ’15). Association for Computing Machinery, New York, NY, USA, Article 29, 16 pages. <https://doi.org/10.1145/2741948.2741962>
- [18] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhanian. 2009. The multikernel: a new OS architecture for scalable multicore systems. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) (SOSP ’09). Association for Computing Machinery, New York, NY, USA, 29–44. <https://doi.org/10.1145/1629575.1629579>
- [19] B. Bershad, T. Anderson, E. Lazowska, and H. Levy. 1989. Lightweight remote procedure call. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles* (SOSP ’89). Association for Computing Machinery, New York, NY, USA, 102–113. <https://doi.org/10.1145/74850.74861>
- [20] Jeff Bonwick. 1994. The slab allocator: an object-caching kernel memory allocator (USTC’94). USENIX Association, USA, 6.
- [21] Silas Boyd-Wickizer, Haibo Chen, Rong Chen, Yandong Mao, Frans Kaashoek, Robert Morris, Aleksey Pesterev, Lex Stein, Ming Wu, Yuehua Dai, Yang Zhang, and Zheng Zhang. 2008. Corey: an operating system for many cores. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) (OSDI ’08). USENIX Association, USA, 43–57.

- [22] Qingchao Cai, Wentian Guo, Hao Zhang, Divyakant Agrawal, Gang Chen, Beng Chin Ooi, Kian-Lee Tan, Yong Meng Teo, and Sheng Wang. [n. d.]. Efficient Distributed Memory Management with RDMA and Caching. 11, 11 ([n. d.]), 1604–1617. <https://doi.org/10.14778/3236187.3236209>
- [23] George Candea, Shinichi Kawamoto, Yuichi Fujiki, Greg Friedman, and Armando Fox. 2004. Microreboot—A Technique for Cheap Recovery. In *6th Symposium on Operating Systems Design and Implementation (OSDI '04)*. USENIX Association, San Francisco, CA.
- [24] J. Chapin, M. Rosenblum, S. Devine, T. Lahiri, D. Teodosiu, and A. Gupta. 1995. Hive: fault containment for shared-memory multiprocessors. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles* (Copper Mountain, Colorado, USA) (SOSP '95). Association for Computing Machinery, New York, NY, USA, 12–25. <https://doi.org/10.1145/224056.224059>
- [25] Matthew Chapman and Gernot Heiser. 2005. Implementing Transparent Shared Memory on Clusters Using Virtual Machines. In *2005 USENIX Annual Technical Conference (USENIX ATC 05)*. USENIX Association, Anaheim, CA. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/implementing-transparent-shared-memory-clusters>
- [26] Jeffrey S. Chase, Henry M. Levy, Michael J. Feeley, and Edward D. Lazowska. 1994. Sharing and Protection in a Single-Address-Space Operating System. *ACM Trans. Comput. Syst.* 12, 4 (November 1994), 271–307.
- [27] David Cheriton. 1988. The V distributed system. *Commun. ACM* 31, 3 (mar 1988), 314–333. <https://doi.org/10.1145/42392.42400>
- [28] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Comput. Surv.* 56, 11, Article 290 (July 2024), 37 pages. <https://doi.org/10.1145/3669900>
- [29] Michal Friedman, Maurice Herlihy, Virendra Marathe, and Erez Petrank. 2018. A persistent lock-free queue for non-volatile memory. (2018), 28–40. <https://doi.org/10.1145/3178487.3178490>
- [30] Jim Gray. 1978. *Notes on Data Base Operating Systems*. Technical Report. Berlin, Heidelberg, 393–481 pages.
- [31] Jinyu Gu, Xinyue Wu, Wentai Li, Nian Liu, Zeyu Mi, Yubin Xia, and Haibo Chen. 2020. *Harmonizing Performance and Isolation in Microkernels with Efficient Intra-Kernel Isolation and Communication*. USENIX Association.
- [32] Gernot Heiser and Kevin Elphinstone. 2016. L4 Microkernels: The Lessons from 20 Years of Research and Deployment. *ACM Trans. Comput. Syst.* 34, 1, Article 1 (apr 2016), 29 pages. <https://doi.org/10.1145/2893177>
- [33] Gernot Heiser, Kevin Elphinstone, Stephen Russell, and Jerry Vochtelloo. 1994. Mungi: A Distributed Single-Address-Space Operating System. In *Proceedings of the 17th Australasian Computer Science Conference (ACSC)*. Christchurch, New Zealand, 271–80.
- [34] Jorrit N. Herder, Herbert Bos, Ben Gras, Philip Homburg, and Andrew S. Tanenbaum. 2007. Failure Resilience for Device Drivers. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*. IEEE, 41–50.
- [35] Yibo Huang, Haowei Chen, Newton Ni, Yan Sun, Vijay Chidambaram, Dixon Tang, and Emmett Witchel. 2025. Tigon: a distributed database for a CXL pod. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (Boston, MA, USA) (OSDI '25). USENIX Association, USA, Article 7, 20 pages.
- [36] Intel Corporation. 2021. eADR: New Opportunities for Persistent Memory Applications. <https://www.intel.com/content/www/us/en/developer/articles/technical/eadr-new-opportunities-for-persistent-memoryapplications.html> Accessed: 2026-03-29.
- [37] Sunita Jain, Nagaradhesh Yeleswarapu, Hasan Al Maruf, and Rita Gupta. 2024. Memory Sharing with CXL: Hardware and Software Design Approaches. arXiv:2404.03245 [cs.ET] <https://arxiv.org/abs/2404.03245>
- [38] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. SeL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) (SOSP '09). Association for Computing Machinery, New York, NY, USA, 207–220.
- [39] Kenneth C. Knowlton. 1965. A fast storage allocator. *Commun. ACM* 8, 10 (oct 1965), 623–624. <https://doi.org/10.1145/365628.365655>
- [40] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a social network or a news media?. In *Proceedings of the 19th International Conference on World Wide Web* (Raleigh, North Carolina, USA) (WWW '10). Association for Computing Machinery, New York, NY, USA, 591–600. <https://doi.org/10.1145/1772690.1772751>
- [41] Seung-seob Lee, Yanpeng Yu, Yupeng Tang, Anurag Khandelwal, Lin Zhong, and Abhishek Bhattacharjee. [n. d.]. MIND: In-Network Memory Management for Disaggregated Data Centers. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (New York, NY, USA, 2021-10-26). Association for Computing Machinery, 488–504. <https://doi.org/10.1145/3477132.3483561>
- [42] Ian Leslie, Derek McAuley, Richard Black, Timothy Roscoe, Paul Barham, David Evers, Robin Fairbairns, and Eoin Hyden. 1996. The Design and Implementation of an Operating System to Support Distributed Multimedia Applications. *IEEE Journal on Selected Areas in Communications* 14 (1996), 1280–1297.
- [43] Kai Li. 1988. IVY: A Shared Virtual Memory System for Parallel Computing. In *International Conference on Parallel Processing*. <https://api.semanticscholar.org/CorpusID:29219727>
- [44] Felix Xiaozhu Lin, Zhen Wang, and Lin Zhong. 2014. K2: a mobile operating system for heterogeneous coherence domains. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems* (Salt Lake City, Utah, USA) (ASPLOS '14). Association for Computing Machinery, New York, NY, USA, 285–300. <https://doi.org/10.1145/2541940.2541975>
- [45] Jing Liu, Yifan Dai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2025. Fast, Transparent Filesystem Microkernel Recovery with Ananke. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 1–18. <https://www.usenix.org/conference/fast25/presentation/liu-jing>
- [46] Arm Ltd. 2024. Arm OCP CMS – Design Considerations in Adding Coherency to CXL Devices (HDM-DB). OCP-CMS Workgroup Presentation. [https://ocp-all.groups.io/g/OCP-CMS/attachment/222/1/Arm%20OCP%20CMS%20-%20Design%20Considerations%20in%20Adding%20Coherency%20to%20CXL%20Devices%20\(HDM-DB\)%20-%20Jan%202012%202024.pdf](https://ocp-all.groups.io/g/OCP-CMS/attachment/222/1/Arm%20OCP%20CMS%20-%20Design%20Considerations%20in%20Adding%20Coherency%20to%20CXL%20Devices%20(HDM-DB)%20-%20Jan%202012%202024.pdf) OCP-CMS meeting, Jan. 12, 2024.
- [47] Jacob Nelson, Brandon Holt, Brandon Myers, Preston Briggs, Luis Ceze, Simon Kahan, and Mark Oskin. 2015. Latency-Tolerant Software Distributed Shared Memory. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*. USENIX Association, Santa Clara, CA, 291–305. <https://www.usenix.org/conference/atc15/technical-session/presentation/nelson>
- [48] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The PageRank Citation Ranking : Bringing Order to the Web. In *The Web Conference*. <https://api.semanticscholar.org/CorpusID:1508503>
- [49] Colby Ranger, Ramanan Raghuraman, Arun Penmetsa, Gary Bradski, and Christos Kozyrakis. 2007. Evaluating MapReduce for Multi-core and Multiprocessor Systems. 2007 IEEE 13th International Symposium on High Performance Computer Architecture, 13–24.
- [50] Andy Rudoff, Chet Douglas, and Tiffany Kasanicky. 2021. Persistent Memory in CXL. <https://www.snia.org/sites/default/files/PMSummit/2021/snia-pm-cs-summit-Rudoff-PM-in-CXL-2021.pdf> SNIA Persistent Memory and Computational Storage Summit.

- [51] Dirk Schmidl, Christian Terboven, Andreas Wolf, Dieter an Mey, and Christian Bischof. 2010. How to Scale Nested OpenMP Applications on the ScaleMP vSMP Architecture. In *2010 IEEE International Conference on Cluster Computing*. 29–37. <https://doi.org/10.1109/CLUSTER.2010.38>
- [52] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiyang Zhang. 2018. LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 69–87. <https://www.usenix.org/conference/osdi18/presentation/shan>
- [53] Jonathan S. Shapiro, Jonathan M. Smith, and David J. Farber. 1999. EROS: A Fast Capability System. In *Proceedings of the Seventeenth ACM Symposium on Operating Systems Principles* (Charleston, South Carolina, USA) (SOSP '99). Association for Computing Machinery, 170–185.
- [54] Anton Shilov. 2025. Samsung, MemVerge, and H3 build 2TB CXL Memory Pool. https://www.anandtech.com/show/20009/samsung_-memverge_-and-h3-build-2tb-cxl-memory-pool.
- [55] Xiang Song, Haibo Chen, Rong Chen, Yuanxuan Wang, and Binyu Zang. 2011. A case for scaling applications to many-core with OS clustering. In *Proceedings of the Sixth Conference on Computer Systems* (Salzburg, Austria) (EuroSys '11). Association for Computing Machinery, New York, NY, USA, 61–76. <https://doi.org/10.1145/1966445.1966452>
- [56] Michael M. Swift, Muthukaruppan Annamalai, Brian N. Bershad, and Henry M. Levy. 2006. Recovering device drivers. *ACM Transactions on Computer Systems* 24, 4 (2006), 333–360.
- [57] Bruce Walker, Gerald Popek, Robert English, Charles Kline, and Greg Thiel. 1983. The LOCUS distributed operating system. In *Proceedings of the Ninth ACM Symposium on Operating Systems Principles* (Bretton Woods, New Hampshire, USA) (SOSP '83). Association for Computing Machinery, New York, NY, USA, 49–70. <https://doi.org/10.1145/800217.806615>
- [58] Qing Wang, Youyou Lu, Erci Xu, Junru Li, Youmin Chen, and Jiwu Shu. 2021. Concordia: Distributed Shared Memory with In-Network Cache Coherence. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 277–292. <https://www.usenix.org/conference/fast21/presentation/wang>
- [59] Zhao Wang, Yiqi Chen, Cong Li, Yijin Guan, Dimin Niu, Tianchan Guan, Zhaoyang Du, Xingda Wei, and Guangyu Sun. 2025. CTXNL: A Software-Hardware Co-designed Solution for Efficient CXL-Based Transaction Processing. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 192–209. <https://doi.org/10.1145/3676641.3716244>
- [60] Zhao Wang, Yiqi Chen, Cong Li, Dimin Niu, Tianchan Guan, Zhaoyang Du, Xingda Wei, and Guangyu Sun. 2025. Enabling Efficient Transaction Processing on CXL-Based Memory Sharing. In *the ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [61] Xingda Wei, Haotian Wang, Tianxia Wang, Rong Chen, Jinyu Gu, Pengfei Zuo, and Haibo Chen. 2023. Transactional Indexes on (RDMA or CXL-based) Disaggregated Memory with Repairable Transaction. arXiv:2308.02501 [cs.DB] <https://arxiv.org/abs/2308.02501>
- [62] David Wentzlaff, Charles Gruenwald, Nathan Beckmann, Kevin Modzelewski, Adam Belay, Lamia Youseff, Jason Miller, and Anant Agarwal. 2010. An operating system for multicore and clouds: mechanisms and implementation (SoCC '10). Association for Computing Machinery, New York, NY, USA, 3–14. <https://doi.org/10.1145/1807128.1807132>
- [63] Emmett Witchel. 2024. Challenges and Opportunities for Systems Using CXL Memory. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 2. <https://doi.org/10.1145/3620666.3655590>
- [64] Lars Wrenger, Florian Rommel, Alexander Halbuer, Christian Dietrich, and Daniel Lohmann. 2023. LLFree: Scalable and Optionally-Persistent Page-Frame Allocation. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 897–914. <https://www.usenix.org/conference/atc23/presentation/wrenger>
- [65] Fangnuo Wu, Mingkai Dong, Gequan Mo, and Haibo Chen. 2023. TreeSLS: A Whole-system Persistent Microkernel with Tree-structured State Checkpoint on NVM. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 1–16. <https://doi.org/10.1145/3600006.3613160>
- [66] Yi Xu, Suyash Mahar, Ziheng Liu, Mingyao Shen, and Steven Swanson. 2024. CXL Shared Memory Programming: Barely Distributed and Almost Persistent. arXiv:2405.19626 [cs.DC] <https://arxiv.org/abs/2405.19626>
- [67] Xiangyao Yu, George Bezerra, Andrew Pavlo, Srinivas Devadas, and Michael Stonebraker. 2014. Staring into the abyss: an evaluation of concurrency control with one thousand cores. *Proc. VLDB Endow.* 8, 3 (nov 2014), 209–220. <https://doi.org/10.14778/2735508.2735511>
- [68] Gerd Zellweger, Simon Gerber, Kornilios Kourtis, and Timothy Roscoe. 2014. Decoupling Cores, Kernels, and Operating Systems. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation* (Broomfield, CO) (OSDI '14). USENIX Association, USA, 17–31.
- [69] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 658–674. <https://doi.org/10.1145/3600006.3613135>
- [70] Xu Zhang, Ke Liu, Yuan Hui, Xiaolong Zheng, Yisong Chang, Yizhou Shan, Guanghui Zhang, Ke Zhang, Yungang Bao, Mingyu Chen, and Chenxi Wang. 2025. DRack: a CXL-disaggregated rack architecture to boost inter-rack communication. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) (USENIX ATC '25). USENIX Association, USA, Article 74, 19 pages.
- [71] Yuhong Zhong, Daniel S. Berger, Pantea Zardoshti, Enrique Saurez, Jacob Nelson, Antonis Psistakis, Joshua Fried, and Asaf Cidon. 2025. My CXL Pool Obviates Your PCIe Switch. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems* (Banff, AB, Canada) (HotOS '25). Association for Computing Machinery, New York, NY, USA, 58–66. <https://doi.org/10.1145/3713082.3730393>
- [72] Xiaowei Zhu, Wenguang Chen, Weimin Zheng, and Xiaosong Ma. 2016. Gemini: A Computation-Centric Distributed Graph Processing System. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Savannah, GA, 301–316. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/zhu>
- [73] Zhiting Zhu, Newton Ni, Yibo Huang, Yan Sun, Zhipeng Jia, Nam Sung Kim, and Emmett Witchel. 2024. Lupin: Tolerating Partial Failures in a CXL Pod. In *Proceedings of the 2nd Workshop on Disruptive Memory Systems* (Austin, TX, USA) (DIMES '24). Association for Computing Machinery, New York, NY, USA, 41–50. <https://doi.org/10.1145/3698783.3699377>

A Component-level Fault-Tolerance Details

In the appendix, we elaborate on how each component of StarfishOS recovers from partial failures:

- Appendix B explains how the memory allocators (i.e., the lock-free buddy system and the slab allocator) recover from partial failures.
- Appendix C explains how the kernel modules other than memory allocators (i.e., IPC, scheduling, and notification) recover from partial failures.
- Appendix D explains how the system services (e.g., the filesystem service) recover from partial failures.

B Memory Allocator Recovery

StarfishOS replaces its original lock-based buddy system with a lock-free allocator called LLFree [64]. This section is structured as follows:

- **LLFree’s Data Structure:** explains LLFree’s data structure,
- **LLFree’s Page Allocation/Free:** explains LLFree’s page allocation/free mechanism, which is also how StarfishOS’s LLFree allocator works.
- **LLFree’s Recovery on StarfishOS:** explains LLFree’s recovery mechanism on StarfishOS.
- **Slab Recovery:** explains the recovery mechanism of the slab allocator used for smaller kernel allocations.

B.1 LLFree’s Data Structure and Memory Organization

LLFree is a lock-free page-level allocator. In its full design, it can support allocating memory sizes in $2^o \times P$ ($o \in \{0, \dots, 10\}$, $P = 4\text{KiB}$), i.e., between 4 KiB and 4 MiB. This is the same as Linux’s buddy system, which employs one buddy allocator per memory zone (e.g., for each NUMA node), supports the orders $o \in \{0, \dots, 10\}$, and uses the base-frame size P as 4KiB.

In this paper, we only rely on LLFree’s natural allocation granularities, i.e., 4 KiB base pages and 2 MiB huge pages, instead of supporting all orders up to 4 MiB. This is because LLFree only supports crash consistency of 4 KiB and 2 MiB pages.

This restriction is sufficient for StarfishOS.

- For user-space memory, cross-machine sharing is triggered by page faults at 4 KiB granularity, so the allocator must support 4 KiB pages.
- Meanwhile, user-space allocators such as libc’s malloc and rpmalloc request large regions from the kernel and manage sub-page objects themselves, rather than relying on the kernel allocator for every small allocation.
- For kernel-space memory, allocations smaller than 4 KiB are already served by the slab system, while the page allocator is only responsible for page-granularity memory.

LLFree Memory Organization. As shown in Figure 17, LLFree organizes each memory zone into a hierarchy of upper level and lower level.

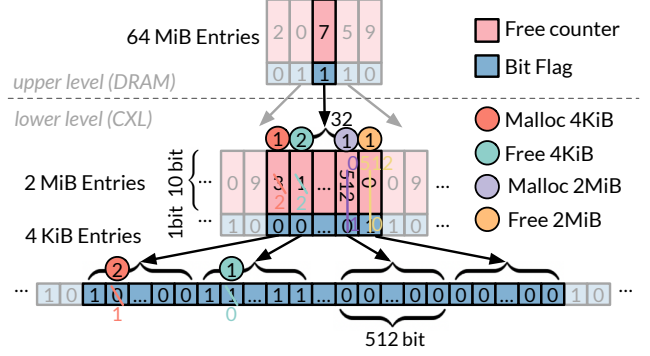


Figure 17: LLFree Allocator

(1) Upper Level. The upper level is mainly used as policy decision making to choose 2MiB entries, we omit the details here for brevity, you can refer to the original paper [64] for more details.

(2) Lower Level. The lower level stores the exact allocation state and is placed in persistent memory for crash consistency. It consists of 2 MiB entries, and 4 KiB entries.

- **2 MiB Entries:** each 2 MiB entry contains a free counter c_L and the allocation flag a . Here, c_L records how many 4 KiB base pages in this 2 MiB region are free, while a indicates whether the whole 2 MiB region has been allocated as one huge page.
- **4 KiB Entries:** each 4 KiB is associated with one bit. A bit value of 0 means the corresponding 4 KiB page is free, and a bit value of 1 means it is allocated.

B.2 LLFree’s Page Allocation/Free

B.2.1 4KiB Page Allocation

A 4KiB page allocation proceeds in two steps:

1. **Decrement the free counter (c_L)** using an atomic operation.
2. **Set a free bit** in the bit field:
 - Scan the bit field to find a zero bit.
 - Atomically set the bit from 0 to 1.

Crash Scenarios.

- **Crash after decrementing c_L but before setting the bit:** The counter underestimates the number of free pages, while the bitfield remains unchanged. This results in a temporary loss of capacity but does not violate correctness.
- **Crash after setting the bit but before decrementing c_L :** The bitfield reflects the allocation, but the counter overestimates free space. This may lead to transient allocation failures but does not cause double allocation.
- **Crash during bit update:** Since bit updates are atomic, the bit is either set or not set. No partial allocation can occur.

Observation. The bitfield is always the ground truth. The free counter may underestimate the number of free pages and cause temporary false out-of-memory decisions, even

though allocatable pages still exist in the bitfield. However, this does not cause double allocation or double free, and can be deterministically repaired from the bitfield during recovery.

B.2.2 Base Page Free

Freeing a base page proceeds in two steps:

1. **Atomically clear the bit** from 1 to 0 in the bitfield.
2. **Increment the free counter** (c_L) using an atomic operation.

Crash Scenarios.

- **Crash after clearing the bit but before incrementing c_L :** The counter underestimates free space. This is safe and can be corrected during recovery.
- **Crash after incrementing c_L but before clearing the bit:** The counter overestimates free space. However, since the bit is still set, no double allocation occurs.
- **Crash during bit update:** Atomicity guarantees that the bit is either cleared or not cleared.

Observation. The bitfield is always the ground truth, while the free counter can be underestimated, which is the same as the observation for 4KiB page allocation.

B.2.3 Huge Page Allocation

A huge page (2MB) allocation is performed using a single atomic compare-and-swap (CAS) on the metadata entry:

- Transition:

$$(c_L = 512, ; a = 0); \rightarrow; (c_L = 0, ; a = 1)$$

Crash Scenarios.

- **Crash before CAS:** No state change occurs.
- **Crash after CAS:** The allocation is fully completed.

Observation. The operation is fully atomic; therefore, no intermediate inconsistent state is observable.

B.2.4 Huge Page Free

Freeing a huge page is performed via a single atomic CAS:

- Transition:

$$(c_L = 0, ; a = 1); \rightarrow; (c_L = 512, ; a = 0)$$

Crash Scenarios.

- **Crash before CAS:** No state change occurs.
- **Crash after CAS:** The free operation is fully completed.

Observation. The operation is fully atomic; therefore, no intermediate inconsistent state is observable.

B.2.5 Summary

- Base page operations are not globally atomic but consist of locally atomic steps.
- Huge page operations are fully atomic via single-word CAS.
- The system guarantees:
 - No double allocation
 - No memory corruption

- Recoverability via bitfield reconstruction

B.3 Recovery of LLFree on StarfishOS

B.3.1 LLFree's Recovery Procedure.

LLFree assumes a stop-the-world failure, so no other allocations or frees run concurrently during recovery. Specifically, the allocator reconstructs metadata as follows:

1. Scan the bitfield of each 2MiB entry.
2. Recompute c_L as the number of zero bits.
3. If c_L is different from the original c_L , CAS the 2MiB entry to use the correct c_L .

B.3.2 StarfishOS's Recovery Procedure.

Challenges. In StarfishOS, we assume a different failure model: one physical machine crashes and recovers, while other machines continue running. The key challenge is that recovery is no longer stop-the-world. While one machine is recovering, other machines may still allocate or free pages in the same memory zone. Therefore, recovery cannot assume that a 2 MiB entry stays unchanged after it is first read.

Solution. To obtain a stable view of the allocator state during recovery, StarfishOS temporarily blocks concurrent allocation/free operations on the affected 64MiB regions. Recovery first sets a global recovery flag in CXL memory and publishes the list of target 64MiB regions to be repaired. The allocation and free fast paths check this flag before modifying allocator metadata. If recovery is active and the target region belongs to the published list, the operation either waits or retries on another eligible region.

A remaining challenge is handling in-flight operations that entered the allocator before the recovery flag was set. To handle this case, each machine waits until all allocator operations that may still access the affected regions have exited their critical sections, and then sends an acknowledgment to the recovery thread. Recovery starts scanning bitfields only after receiving acknowledgments from all live machines. This guarantees that no concurrent operation can modify the target regions while recovery is recomputing and repairing their counters.

B.4 Slab Recovery

LLFree in StarfishOS only handles page-granularity CXL allocation. Smaller kernel allocations still use per-CPU slab instances built on top of those pages. Since each core has its own slab instance, partially concurrent allocations/frees on slab instance will not block other instance. During recovery, the slab's lock is reconstructed.

Still, we need to ensure that partly conducted allocation-s/frees on slab instance will leave the slab instance in a consistent state. To achieve this, we use a lightweight in-memory undo record for each slab instance.

Per-slab undo record. Each slab header stores the metadata needed to manage one slab, including:

- `free_list_head`: pointer to the first free object;

- `free_cnt`: number of free objects in the slab;
- lock and list-link fields used by the allocator’s normal fast path.

To make updates recoverable, we extend the header with a small in-flight undo record that contains:

- the operation type (Alloc, Free, or None);
- the old free-list head;
- the old free-object count.

Conceptually, the extended header is:

```
struct slab_header { free_list_head; free_cnt;
...; undo{ op, old_free_list_head, old_free_cnt
}; }
```

The undo record is persisted before the allocator mutates the slab header.

C Kernel-side Recovery for IPC, Scheduler, and Notification

This section presents the recovery mechanisms for StarfishOS’s shared kernel-side coordination structures, which include: a lock-free queue (§C.1) used by IPC, Scheduler, and Notification, together with the recovery metadata carried by queue nodes.

C.1 Lock-Free Queue with Recovery Metadata

In StarfishOS, cross-machine IPC, thread scheduling, and event notification all share a common communication pattern: **Multi-Producer Single-Consumer (MPSC)**. The design is inspired by the DurableQueue proposed by Friedman et al. [29], and StarfishOS extends the data structure and algorithms to fit the OS-level MPSC architecture and our specific failure model.

Data Structure. Our queue nodes embed a status and a customizable payload defined by specific use cases.

Unified Data Structure for StarfishOS Queue.

```
Node {
Node* next;
char status ∈ {INIT, DOING, DONE, ABORT};
Payload payload; /* Defined by specific use cases
(IPC/Sched/Noti) */
}
Queue { Node* head; Node* tail; }
```

Enqueue (Producer). The enqueue operation (Algorithm 1) remains lock-free. A producer:

1. Allocates a node with status = INIT, embeds the request in the payload, and FLUSH(node) (Line1).
2. Link it into the list with CAS (Line6).
3. Flush the linking pointer (Line7).
4. Change tail to the new node (Line8).

The lock-free nature ensures that if a producer crashes mid-enqueue, other producers can help finish the linking (Lines 11-12) and continue their own operations.

- **Linearization point:** successful CAS(&last->next, next, node).
- **Crash tolerance (producer):** if an enqueueer fails after publishing last->next but before advancing tail, other producers will help advance tail.

Algorithm 1: enq(payload) for StarfishOS.

```
1 node ← new Node(INIT, payload); FLUSH(node)
2 while true do
3   last ← tail; next ← last->next
4   if last == tail then
5     if next == NULL then
6       if CAS(&last->next, next, node) then
7         FLUSH(&last->next)
8         CAS(&tail, last, node)
9         return node
10    else
11      /* Help crashed enqueueer */
12      FLUSH(&last->next)
13      CAS(&tail, last, next)
```

Dequeue (Consumer). The dequeue operation (Algorithm 2) is executed exclusively by the single consumer (e.g., the target CPU core). When queue is not empty, the consumer:

- 1.Dequeues the node (Line9).
2. Transitions its status from INIT to DOING (Line10).
3. Processes the payload via a use-case-specific handle() function (Line12).
4. Marks it as DONE (Line13) and persists it (Line14).

Line7 fix inconsistency caused by enqueueer crash.

Recovery. The status field serves as queue-level recovery metadata that records how far a request has progressed and guides post-crash handling. Under a producer crash, recovery determines whether the request was ever published to the queue; under a consumer crash, recovery determines whether handling had not started, completed, or stopped in an ambiguous intermediate state.

- **Producer crash:** if the producer fails before linking the node, the request is absent. If it fails after linking the node, the request remains visible in the queue and the consumer can still handle it. Thus, producer failure never corrupts the queue and only determines whether the request was successfully published.
- **Consumer crash:** recovery interprets the node through status, which tells the client what it may safely conclude: INIT means the request was published but not yet started; DONE means handling completed and the response is valid; DOING means the request may have stopped after partial execution; and ABORT records an explicit failed/canceled outcome.

Algorithm 2: deq() for StarfishOS.

```
1 while true do
2   first ← head; last ← tail; next ←
   first->next
3   if first == last then
4     if first == last then
5       if next == NULL then
6         return NULL
7       FLUSH(&last->next); CAS(&tail, last,
   next)
8   else
9     n ← next
10    if CAS(&n->status, INIT, DOING) then
11      FLUSH(&n->status)
12      handle(n) /* Varies by use case
   (IPC/Sched/Noti) */
13      n->status ← DONE
14      FLUSH(&n->status)
15      CAS(&head, first, next)
16    return
```

C.2 Applications of the DurableQueue

We instantiate the DurableQueue for three core OS components.

C.2.1 IPC Queue

For inter-machine IPC, the queue serves as a message channel. The producer allocates a variable-sized node to accommodate the msgBuffer. After enqueueing, the producer spins on n->status. Once the consumer executes the message and sets status = DONE, the producer reads the response from msgBuffer. The consumer handles the message based on the msgType, it writes the response back to msgBuffer, and persists the msgBuffer during handle(n).

- **Payload:** msgType, msgBuffer (variable-sized).
- **Queue:** IPCQueue ipcQueue[MAX_MACHINE_NUM].
- **Producer:** applications or system services that invoke IPC.
- **Consumer:** the target machine's user-space server thread, which is invoked by the interrupt-and-polling mechanism described in the main text.
- **Producer crash:** in cases of client crash, uncommitted request will be lost, while committed request will eventually be handled by the server. After recovery, the client can check status to determine whether the request is committed or not.
- **Consumer crash:** Consumer (i.e., system service server) crash happens at three cases: (1) server not yet started the request (INIT), (2) server started the request but not yet completed (DOING), (3) server completed the request but not yet persisted (DONE). Producer (i.e., client) can timeout

when waiting for server's response, and the client will set status to ABORT to notify the server to abort this request. After server recovery, the server can check status to determine whether the request is committed or not. For INIT request, the server can simply retry the request. For DOING request, the server can simply abort or redo the request (changes of the request are already undo since service recovery will only replay committed request) and set status to ABORT or DONE. For DONE/ABORT request, the server simply skip the request.

C.2.2 Scheduler Queue

Each CPU core has its own SchedQueue, in total, StarfishOS maintains a global SchedQueue schedQueues[MAX_CPU_NUM] on CXL. When a thread needs to be migrated to a remote CPU (due to sched_setaffinity or load balancing), the enqueueer pushes a node containing the Thread* payload. The target CPU periodically polls its SchedQueue during its scheduling tick. The handle(n) function simply takes the Thread* and enqueues it into the CPU's local volatile runqueue.

- **Payload:** Thread*.
- **Queue:** SchedQueue schedQueues[MAX_CPU_NUM].
- **Producer:** kernel scheduler.
- **Consumer:** target CPU core, dequeues during looping local queues to schedule threads.
- **Producer crash:** Under producer crash, the shared queue itself remains consistent and other machines can still enqueue threads to that CPU because the queue is lock-free. At worst, the scheduling attempt issued by the failed machine is lost. This is equivalent to failure of the scheduling thread crash on the failed machine.
- **Consumer crash:** Under consumer crash, consumer's machine is crashed together with all core's private queues state lost. Other CPUs are unaffected because each CPU has its own shared queue. After recovery, the scheduler reconstructs each core's private queue and re-enqueue any interrupted DOING entries.

C.2.3 Notification Queue

Notification queues (e.g., semaphores) manage waiting lists. For example, in a MapReduce application, the main thread calls sem_wait (invoke sys_wait syscall), enqueueing its Thread* into the waiting list. When worker threads finish, the last worker calls sem_post (invoke sys_notify syscall), which acts as the dequeuer. The handle(n) function wakes up the main thread. If a waiting thread (enqueueer) is killed abruptly, its lock-free enqueue ensures the semaphore's waiting list is not corrupted, allowing other threads to continue waiting or posting.

- **Payload:** Thread* (waiting thread), cpu_id (waiting thread's CPU ID).
- **Queue:** NotiQueue notiQueues for each application defined notification, i.e., a semaphore variable of type sem_t.

- **Producer/consumer:** threads invoking `sys_wait` and `sys_notify` on the shared waiting list.
- **Producer crash:** if a waiter fails before publishing its node, it is absent from the waiting list; if it fails after publishing, the node remains visible but the waiting thread itself may already be gone. In both cases, the waiting-list metadata remains consistent.
- **Consumer crash:** if the notifier side fails while waking a waiter, INIT means the wake-up has not started, while DOING means the wake-up may have been interrupted and is treated as failed during recovery. If the associated cross-machine process partially fails, the process is reclaimed and the notification becomes unusable; DurableQueue only ensures that these failures do not corrupt kernel metadata or crash other threads invoking `sys_wait` or `sys_notify`.

D Filesystem Instance Recovery with Ananke-style P-log

This section describes the filesystem recovery mechanism briefly introduced in § 6.2. In StarfishOS, each filesystem instance manages a disjoint subset of the namespace (e.g., a mount point). Thus, when the machine hosting such an instance fails, another machine must recreate that instance to restore access to the affected files.

Basic idea. StarfishOS follows Ananke’s micro-reboot style recovery [45]. Each filesystem instance maintains a persistent operation log (p-log) in CXL shared memory. Mutating operations append a redo log entry before replying to the client. If the instance’s host machine crashes, a surviving machine starts a replacement instance, replays the p-log, and then resumes serving requests for the same namespace partition.

What is logged. The p-log only records filesystem operations whose effects must survive the failure of the original instance, such as file creation and writes. The log is append-only and persisted in entry order, so recovery can replay only complete entries and ignore any partially appended tail entry left by the crash. The log contains a small persistent header and a sequence of variable-sized entries:

- `plog_header`: log state, tail offset, and sequence metadata;
- `plog_entry`: operation type plus the operation payload.

For the filesystem operations used in StarfishOS, each entry stores enough information to replay the operation on a fresh instance, e.g., pathname and permission bits for file creation, or pathname, offset, and data payload for a write.

Recovery procedure. Recovery consists of three steps:

1. Start a new filesystem instance on a surviving machine.
2. Map the failed instance’s p-log from CXL shared memory and replay the logged operations to reconstruct the filesystem state.

3. Register the new instance as the owner of the failed namespace partition by updating the filesystem coordination metadata described in § 6.1.

Operationally, the replacement instance rebuilds in-memory filesystem state from the replayed entries and only then begins serving new requests for that namespace partition.

Relation to IPC recovery. This mechanism restores the filesystem instance itself; it does not make an interrupted IPC appear to have completed successfully. As discussed in § 5.1, after a crash, requests that were in flight are reported to clients as failed or aborted, and clients reconnect to the recovered instance and retry if appropriate. Because mutating operations are logged before the service replies, any filesystem update already reported as complete to the client is preserved across instance recovery even if it has not yet reached SSD, while IPC still exposes failure explicitly at request granularity.